PROJETO 3

JEFFERSON LUIS NAMUR JUNIOR, LUIZ GUSTAVO BECHER BOLZANI, ZOLTAN GEOCZE

INTRODUÇÃO

Este trabalho tem como objetivo implementar e comparar algoritmos de ordenação, analisando seu desempenho teórico e prático. Foram utilizados os algoritmos Merge Sort e Quick Sort, avaliando seu comportamento em diferentes cenários de entrada.

FUNDAMENTAÇÃO TEÓRICA

O Merge Sort é um algoritmo de ordenação baseado na técnica de divisão e conquista, que divide o vetor em partes menores, ordena essas partes e posteriormente as combina.

O Quick Sort também utiliza divisão e conquista, escolhendo um pivô e reorganizando os elementos menores e maiores em relação a ele.

ANÁLISE ASSINTÓTICA

- Merge Sort:
  - Melhor caso: O(n log n)
  - Caso médio: O(n log n)
  - Pior caso: O(n log n)
- Quick Sort:
  - Melhor caso: O(n log n)
  - Caso médio: O(n log n)
  - Pior caso: $O(n^2)$

METODOLOGIA

Foram realizados experimentos com vetores de tamanhos:

- 1000
- 5000
- 10000

Os cenários analisados foram:

- Melhor caso: vetor já ordenado
- Caso médio: vetor com valores aleatórios
- Pior caso: vetor ordenado de forma inversa

Cada experimento foi executado 30 vezes, medindo o tempo em nanossegundos utilizando System.nanoTime().

RESULTADOS

```
===== PROJETO 3 - ORDENACAO =====

Tamanho do vetor: 1000 | Melhor Caso
Merge Sort -> Media: 62216,67 ns | Desvio Padrao: 118637,85 ns
Quick Sort -> Media: 687116,67 ns | Desvio Padrao: 1096473,27 ns

Tamanho do vetor: 1000 | Caso Medio
Merge Sort -> Media: 105863,33 ns | Desvio Padrao: 45665,64 ns
Quick Sort -> Media: 58423,33 ns | Desvio Padrao: 31645,58 ns

Tamanho do vetor: 1000 | Pior Caso
Merge Sort -> Media: 63486,67 ns | Desvio Padrao: 3276,56 ns
Quick Sort -> Media: 275656,67 ns | Desvio Padrao: 9245,92 ns

--------------------------------------------------
Tamanho do vetor: 5000 | Melhor Caso
Merge Sort -> Media: 174900,00 ns | Desvio Padrao: 15829,99 ns
Quick Sort -> Media: 9854913,33 ns | Desvio Padrao: 208507,61 ns

Tamanho do vetor: 5000 | Caso Medio
Merge Sort -> Media: 414820,00 ns | Desvio Padrao: 123661,11 ns
Quick Sort -> Media: 224026,67 ns | Desvio Padrao: 13929,10 ns

Tamanho do vetor: 5000 | Pior Caso
Merge Sort -> Media: 263550,00 ns | Desvio Padrao: 268824,98 ns
Quick Sort -> Media: 7271270,00 ns | Desvio Padrao: 123136,69 ns

--------------------------------------------------
Tamanho do vetor: 10000 | Melhor Caso
Merge Sort -> Media: 272106,67 ns | Desvio Padrao: 18467,45 ns
Quick Sort -> Media: 39309556,67 ns | Desvio Padrao: 583068,81 ns

Tamanho do vetor: 10000 | Caso Medio
Merge Sort -> Media: 739256,67 ns | Desvio Padrao: 49125,69 ns
Quick Sort -> Media: 459706,67 ns | Desvio Padrao: 6291,63 ns

Tamanho do vetor: 10000 | Pior Caso
Merge Sort -> Media: 438226,67 ns | Desvio Padrao: 28161,14 ns
Quick Sort -> Media: 28819286,67 ns | Desvio Padrao: 408719,51 ns

--------------------------------------------------
```

DISCUSSÃO

Os resultados demonstraram que o Merge Sort apresentou desempenho estável em todos os cenários, conforme esperado teoricamente.

Já o Quick Sort apresentou excelente desempenho no caso médio, porém teve queda significativa no pior caso. Isso ocorreu porque o algoritmo utilizou o último elemento como pivô, o que faz com que vetores já ordenados ou inversamente ordenados resultem em partições desbalanceadas.

Essa característica levou ao aumento do tempo de execução, aproximando-se da complexidade $O(n^2)$, o que foi observado nos experimentos.

Além disso, pequenas variações nos tempos podem ser explicadas por fatores como otimizações da JVM, uso de cache do processador e precisão da medição em nanossegundos.

CONCLUSÃO

Os resultados indicam que o Merge Sort apresenta maior estabilidade de desempenho, enquanto o Quick Sort pode ser mais eficiente em casos médios, porém depende da escolha do pivô.

Dessa forma, a escolha do algoritmo deve considerar o tipo de entrada e a necessidade de previsibilidade no desempenho.